



Subscription Tracking Application

STArt or SUBsTRACT

Mini-Project Overview

Hey Team,

Here is the complete breakdown of our mini-project idea: a Subscription Tracking Application. Please read through this today so we are all on the same page before we present the deep-dive to our project guide tomorrow.

1. What is it and why are we building it?

The average person has multiple subscriptions (JioHotstar, Netflix, Spotify, Gym, Cloud Storage, etc.) but often forgets when they renew, leading to wasted money. Our web application will allow users to manually log all their subscriptions in one place, view their total monthly/yearly expenses, and see upcoming renewal dates.

Why it is a great mini-project:

- It demonstrates User Authentication, CRUD (Create, Read, Update, Delete) operations, relational database design, and complex date-time logic.

2. Core Features (Scope of Work)

To ensure we finish on time, we need to be strict about our scope. The project is divided into must-have Phase 1 functionality and optional Phase 2 enhancements.

Phase 1: Must-Have Features (Core)

- User Authentication: Secure Login and Registration (passwords must be hashed).
- CRUD Operations: Users can Add, View, Edit, and Delete their subscriptions.
- Dashboard Analytics: A main page showing:
 - Total Monthly Spend (calculated by aggregating monthly + (yearly/12) costs).
 - Total Yearly Spend.
 - A list of "Upcoming Renewals" sorted by the nearest date.

Phase 2: Good-to-Have Features (If time permits)

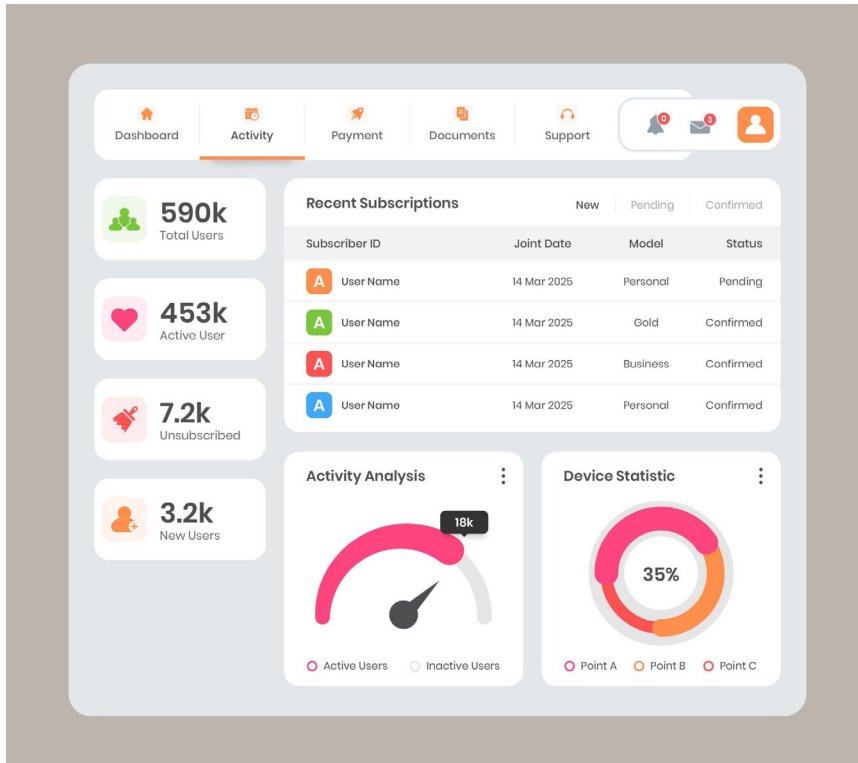
- Visual Data: A pie chart showing spending by category (e.g., Entertainment vs. Software).
- Notifications: An email alert sent 3 days before a subscription renews.

🚫 Out of Scope (What we will AVOID)

- No real payment gateways: We will NOT integrate Stripe or PayPal. We are tracking expenses, not processing them.
- No live bank syncing: We will rely on manual user input to keep the architecture manageable.

3. How It Works (The User Flow)

1. Signup/Login: User creates an account.
2. Empty State: User sees a clean dashboard prompting them to "Add a Subscription."
3. Data Entry: User enters: Service Name (Netflix), Cost (₹649), Cycle (Monthly), and Next Renewal Date.
4. Processing: The backend calculates the metrics and saves the data.
5. Dashboard View: User is redirected to the dashboard, which now displays their financial metrics and upcoming bills.



4. Technology Stack Options

We need to finalize our stack before meeting the guide. Here are the best options for us:

Option A: The Classic Java Stack (Recommended)

- Frontend: HTML, CSS (Bootstrap for quick styling), JSP (JavaServer Pages).
- Backend: Java Servlets (Tomcat Server).
- Database: MySQL.

Pros: Highly respected by examiners, demonstrates strong understanding of MVC architecture and relational databases.

Option B: The MERN Stack (Modern)

- Frontend: React.js, Tailwind CSS.
- Backend: Node.js, Express.js.
- Database: MongoDB.

Pros: Industry standard, great for resumes, NoSQL database allows flexible data structures.

Option C: Python / Django

- Frontend: HTML, CSS, Django Templates.
- Backend: Python (Django framework).
- Database: SQLite (default) or MySQL.

Pros: Very fast development time, Python makes date-time and math calculations very easy.

Recommended approach: Build the web application first, then add PWA support for Android-like installability. If extra time remains, consider a cross-platform mobile wrapper using Flutter or React Native.

5. Mobile Strategy: PWA vs Cross-Platform Android

Our guide asked whether the project should be a PWA or a cross-platform solution that works as both a web application and an Android app. Here is the comparison:

Progressive Web App (PWA)

- Works as a responsive web application in the browser. Users can install it from the browser on Android.
- Supports offline caching, install prompt, and push notifications with service workers.
- Uses one codebase for web and mobile-like behavior, which reduces development time.

Cross-Platform Android App

- Built with Flutter or React Native for a native-like Android experience.
- Offers better access to device features, smoother animations, and Play Store distribution.
- Requires a separate mobile app build, but still can share the same backend API.

Recommendation:

For our mini-project, the best path is to implement a responsive web application first, then add PWA support. This satisfies both the web and Android requirement with the smallest scope. If time allows, a lightweight cross-platform wrapper can be added later as an optional extension.

6. API Implementation Requirement

Our guide also asked us to add at least one API implementation. This is a strong requirement and a great way to show real backend capability. Here is the API plan:

- ✓ Implement a REST API for authentication and subscription management.
- ✓ Example endpoints:
- ✓ POST /api/auth/register — create user accounts.
- ✓ POST /api/auth/login — authenticate users and return tokens.
- ✓ GET /api/subscriptions — list the user's subscriptions.
- ✓ POST /api/subscriptions — add a new subscription.
- ✓ PUT /api/subscriptions/{id} — update an existing subscription.
- ✓ DELETE /api/subscriptions/{id} — remove a subscription.
- ✓ GET /api/dashboard — calculate totals and upcoming renewal data.

For cost and working possibility, the API is very feasible and low-cost to implement for this mini-project. The API can be built in any of the recommended stacks: Java, Node.js, or Django. The backend should return JSON, and the web frontend can consume it easily.

7. Costing, Working and Possibility

To make a practical case, here is the expected cost and feasibility picture for this mini-project:

- Development effort: 1-2 weeks for a working web app with API and dashboard.
- Hosting cost: Use free or very low-cost services like PythonAnywhere free tier, Render free tier, Heroku free tier, or a basic shared hosting plan.
- Tools: The project can be developed with free tools (VS Code, GitHub, free database tiers).
- PWA support: Provided by the same web app with no separate mobile backend, so cost remains low.

Possible implementation approaches:

- ❖ A Django app with REST API and responsive templates is simplest and quickest.
- ❖ A Node.js/Express backend with React frontend is more modern but slightly more work.
- ❖ A Java backend with JSP is exam-friendly and strong for relational database design.

Overall possibility:

This mini-project is highly feasible. The primary work is building the subscription CRUD flows, dashboard calculations, and a simple API. Adding PWA support is optional but recommended. A full Android wrapper is possible if there is extra time.

8. Recommended Features and Enhancements

These features make the application more complete and presentable:

- ✚ Category tags for subscriptions (Entertainment, Software, Utilities, Education).
- ✚ Search and filter by subscription name, category, or renewal date.
- ✚ Custom billing cycle support beyond monthly/yearly (quarterly, weekly, custom).
- ✚ Export subscriptions to CSV or PDF.
- ✚ A summary card for "Next 30 days" renewals.
- ✚ Optional email reminders or in-app alert banners.

These enhancements can be added in Phase 2 once the core app is stable.

9. Architecture and Data Flow

Recommended architecture for a clean mini-project:

- Frontend: Responsive web pages or PWA, consuming API endpoints.
- Backend API: REST endpoints for auth and subscription CRUD.
- Database: Relational store for users and subscriptions.

Data flow summary:

- User enters data in the web app or PWA.
- Frontend sends JSON requests to the API.
- Backend validates and stores data in the database.
- Dashboard endpoint computes totals and returns results.
- Frontend renders metrics and renewal alerts.

10. Next Steps for Our Team

Before the next meeting, we should decide and assign:

- Which technology stack we will use: Java, MERN, or Django.
- Whether we will target a PWA or also build a cross-platform Android wrapper.
- Which API endpoints are required for the first working demo.
- How we will split frontend, backend, and presentation tasks among the team.

11. Demo Flow and Presentation Summary

Use this section to guide your presentation and make the demo easy to follow for the project guide.

- ✚ Start with the problem: multiple subscriptions are hard to track, renewals are missed, and users lose money.
- ✚ Show the login/sign-up page and explain secure authentication.
- ✚ Demonstrate adding a subscription and describe how the backend stores it.
- ✚ Show the dashboard with total monthly/yearly spend and upcoming renewals.
- ✚ Explain the API endpoints and how the frontend uses them, especially for PWA/mobile compatibility.
- ✚ Wrap up by describing the mobile strategy: PWA first, optional Android wrapper later.

Feature Priority Table

Priority	Feature	Why it matters
High	User Authentication	Secures data and shows examiner understanding of auth.
High	Subscription CRUD	Core app functionality and primary user value.
High	Dashboard Analytics	Demonstrates data processing and useful output.
Medium	API Implementation	Required by guide and enables mobile/web integration.
Medium	PWA Support	Makes the app installable on Android without a separate app.
Low	Email Notifications	Nice enhancement if time permits.

Slide-style Summary Page

Use these short, strong statements to present the project quickly and clearly.

Subscription Tracking Application

STArt or SUBsTRACT

Web + PWA-ready, API-driven, exam-ready.

- Core focus: secure login, subscription management, and renewal analytics.
- API backend enables web app and mobile-compatible client integration.
- PWA support gives Android installability with one shared codebase.
- Optional Android wrapper is feasible later if extra time remains.
- Presentation focus: problem, demo flow, API, and mobile strategy.